



SALI

Super Agent Liquidity & Risk Intelligence Platform

Complete Technical Documentation

[Codex Community Hackathon](#) | [bKash presents SUST CSE Carnival 2026](#)

All data is SYNTHETIC | No real wallets, credentials, or customer data

Contents

1.	Executive Summary	3
2.	Problem & Solution	4
3.	System Architecture	5
4.	Tech Stack	6
5.	User Roles & Access Control	7
6.	Coordination Workflow	9
7.	Unified Liquidity — How Values Are Calculated	11
8.	Anomaly Detection Engine	14
9.	AI Assistant & Advisory System	16
10.	Demo Scenarios (A/B/C/D)	17
11.	Responsible Design Guardrails	19
12.	Deployment Guide	20

1. Executive Summary

SALI (Super Agent Liquidity & Risk Intelligence Platform) is a decision-support platform for multi-provider mobile financial service super agents in Bangladesh. It serves agents who operate across three providers (bKash, Nagad, Rocket) from a single physical cash drawer but maintain three separate e-money balances.

The platform unifies the view of physical cash and per-provider e-money, forecasts shortages before they disrupt service, detects unusual transaction patterns with explainable evidence, and routes alerts through a safe, traceable human coordination workflow. It never executes real financial transactions, never declares fraud, and always preserves provider boundaries.

The system supports five user roles (Super Agent, Field Officer, Area Manager, Risk Analyst, Management) with strict role-based access control. Each role sees a different scope of data and has different action permissions, enforced at both the frontend and API levels.

Key Capabilities

Capability	Description
Unified Liquidity View	Shared cash + 3 provider e-money displayed separately, never merged
Shortage Forecasting	Burn-rate projection with ETA to 20% threshold, confidence-adjusted
Anomaly Detection	3 detectors: repeated amounts, velocity spikes, data-quality conflicts
AI Advisory	Bengali & English alert explanations using OpenAI GPT-4o-mini
Coordination Workflow	Acknowledge -> Assign -> Escalate -> Resolve with full audit trail
Role-Based Access	5 roles with scoped data, action permissions, and view gating
Real-Time Updates	Socket.io mini-service auto-ticks data every 9 seconds
Voice I/O	Text-to-speech (OpenAI TTS) + speech-to-text (Web Speech API)

2. Problem & Solution

The Problem

Super agents serve customers across bKash, Nagad, and Rocket from one shared cash drawer, but each provider has a separate e-money balance. They cannot easily answer: "Will I have enough cash and provider balance to serve customers for the next few hours?" The situation worsens when most pressure comes from one provider, or when unusual transaction patterns appear that may be normal Eid demand or may need review.

Additionally, the agent, field officer, area manager, and risk reviewer may not know who should act first or how to coordinate until the issue is resolved.

The Solution

SALI provides a unified operational view that shows physical cash and each provider's e-money separately, forecasts shortages with confidence levels, detects unusual activity with evidence and uncertainty, and routes alerts through a structured coordination workflow with clear ownership and audit trails.

Provider Boundaries

The platform shows one combined outlet view but never implies that one provider can control another's balance, data, or decisions. Cash and e-money are displayed side by side but never merged. Cross-provider conversion is explicitly forbidden and not supported.

3. System Architecture

The system has four layers: Users & Roles, Frontend (Next.js), Backend (API Routes), and Data & Realtime.

Layer	Component	Technology
Users & Roles	5 role-based logins with scoped access	Zustand + localStorage
Frontend	12 views, single-route SPA	Next.js 16, React 19, Tailwind 4, shadcn/ui
Backend	16 API routes (REST)	Next.js API Routes, Prisma ORM
AI Services	Assistant, Advisory, TTS	OpenAI GPT-4o-mini, OpenAI TTS-1
Realtime	Auto-tick + anomaly scan	Socket.io mini-service (port 3001)
Database	Agents, balances, transactions, alerts, cases	PostgreSQL (Neon) / SQLite (local)

Data Flow

1. The realtime engine ticks every 9 seconds, generating transactions and updating balances.
2. Every 35 seconds, the anomaly scan runs across all agent/provider pairs.
3. New alerts are created with full evidence (facts, sample transactions, uncertainty, false-positive note).
4. Each alert auto-creates a coordination case routed to the appropriate role.
5. The frontend receives a Socket.io broadcast and re-fetches data from the API.
6. Users see live-updating KPIs, alert feeds, and balance charts.

4. Tech Stack

Category	Technology	Purpose
Framework	Next.js 16 (App Router)	Full-stack framework with API routes
Language	TypeScript 5	Type safety throughout
Styling	Tailwind CSS 4 + shadcn/ui	Dark fintech theme, responsive design
Database	PostgreSQL (Neon) / SQLite	Agents, balances, transactions, alerts, cases
ORM	Prisma 6	Type-safe database access
Realtime	Socket.io	Live dashboard updates
Charts	Recharts	Area charts, bar charts, sparklines
State	Zustand + persist middleware	Client state, localStorage persistence
AI	OpenAI GPT-4o-mini + TTS-1	AI Assistant chat, advisory generation, voice output
Icons	Lucide React	Consistent icon system
Fonts	Geist Sans/Mono + Hind Siliguri	Latin + Bengali text rendering

5. User Roles & Access Control

The platform enforces a strict hierarchy: Agent -> Field Officer -> Area Manager -> Risk Analyst. Management sits above as a read-only observer. Each role has different data scope, view access, and action permissions, enforced at both the frontend (view gating, button disable) and API (permission checks, status transition validation, agent ownership verification).

Role	Persona	Scope	Ack	Assign	Escalate	Resolve	Risk Revi
Super Agent	Karim Sheikh (AG-KAR-001)	Own outlet only	Yes	No	Yes	No	No
Field Officer	Tanvir Ahmed, Dhaka North	Assigned territory	Yes	No	Yes	Yes	No
Area Manager	Nadia Rahman, Dhaka North	Full area/district	Yes	Yes	Yes	Yes	No
Risk Analyst	Sadia Karim (reviewer)	Network (review only)	Yes	No	Yes	No	Yes
Management	Rezaul Karim (Head)	Aggregate (read-only)	No	No	No	No	No

Key Differentiators

Super Agent: Only role scoped to a single outlet. Sees only their own data. Can acknowledge and escalate but cannot resolve or assign. Must sign out to switch roles.

Field Officer: First responder who contacts outlets and reviews on the ground. Can resolve if fixed at outlet level but cannot assign cases to others.

Area Manager: Only role that can assign cases to field officers. Full coordination access. Approves authorised support (provider refills).

Risk Analyst: Only role that can submit Risk Reviews (formal verdict on anomaly evidence). Reviews but never closes cases. Sees cross-provider Relationship Graph.

Management: Read-only. Sees aggregate KPIs only. Cannot act on individual cases.

6. Coordination Workflow

The coordination workflow enforces a strict state machine. Cases move through statuses in order, and invalid transitions are rejected by the API with a 403 error.

Status Flow

```
Open -> Acknowledged -> Assigned -> Escalated -> Resolved
|-> Escalated -> Resolved (can skip Assigned)
|-> Resolved (can close directly from Acknowledged)
Resolved -> Open (reopened by any role with acknowledge permission)
```

Transition Rules (enforced server-side)

Current Status	Allowed Actions	Blocked Actions
Open	Acknowledge, Note, Risk Review	Escalate, Resolve, Assign, Reopen
Acknowledged	Assign, Escalate, Resolve, Note, Risk Review	Acknowledge (already done), Reopen
Assigned	Escalate, Resolve, Note, Risk Review	Acknowledge, Assign (already done), Reopen
Escalated	Resolve, Note, Risk Review	Acknowledge, Assign, Escalate (already done), Reopen
Resolved	Reopen, Note, Risk Review	All other actions

Example: Scenario A Walkthrough

- Alert fires (Open):** Nagad e-money at AG-KAR-001 draining at 38,000 BDT/h. Projected shortage in ~1.5h.
- Agent acknowledges (Open -> Acknowledged):** Karim sees the alert on his outlet dashboard, clicks Acknowledge. Ownership updates to 'Karim Sheikh (Agent)'. Audit trail logs the event.
- Agent escalates (Acknowledged -> Escalated):** Karim can't arrange a refill himself. Clicks Escalate. Case appears as 'Escalated' on Field Officer's board.
- Field Officer responds:** Tanvir reviews, contacts outlet, adds note. Escalates to Area Manager for authorised support.
- Risk Analyst reviews:** Sadia examines evidence, submits Risk Review verdict: 'Likely normal demand, FP risk HIGH'. She cannot resolve the case.
- Area Manager resolves (Escalated -> Resolved):** Nadia reads the review, approves authorised Nagad refill, clicks Resolve with closing note. Case closed.

7. Unified Liquidity — How Values Are Calculated

The 4 Balance Cards

Each card represents one pool of money. They are displayed side by side but never merged.

Card	Source	Capacity	What It Means
Physical Cash (shared)	Agent.cashBalance	800,000 BDT	Cash in the drawer. Shared across all providers.
bKash e-Money	AgentProviderBalance.balance	500,000 BDT	Separate bKash digital balance.
Nagad e-Money	AgentProviderBalance.balance	500,000 BDT	Separate Nagad digital balance.
Rocket e-Money	AgentProviderBalance.balance	500,000 BDT	Separate Rocket digital balance.

Percentage of Capacity

```
pct = balance / capacity
```

```
Green bar if >= 40%, Amber if 20-40%, Red if < 20%
```

```
The 20% threshold is the 'shortage' trigger – if balance drops below this, a critical alert is generated.
```

Burn Rate Calculation

The burn rate measures how fast a balance is changing, based on BalanceSnapshot records from the last 60 minutes:

```
burnRatePerHour = (firstSnapshot.balance - lastSnapshot.balance) / hoursBetweenThem
```

Positive = draining (money going out). Negative = building (money coming in). Requires at least 2 snapshots in the lookback window. If fewer than 2, burn rate = 0.

Shortage Projection (Hours to Shortage)

If the balance is draining (burn rate > 0), the system projects when it will cross the 20% threshold:

```
threshold = capacity x 0.20
```

```
hoursToShortage = (currentBalance - threshold) / burnRatePerHour
```

Only shown if burn rate > 0 and hoursToShortage > 0 and < 72 hours. Severity: Critical if <= 1h, High if <= 3h, Warning if > 3h.

If current balance is already below the threshold, the shortage is flagged as 'critical' immediately.

Confidence Calculation

Each provider feed has a confidence value (0.0 to 1.0) stored in the database. The forecast engine adjusts it:

```
if feed is stale (isStale=true AND latencyMs > 60s):
```

```
confidence = min(confidence, 0.45) -> capped at Low
```

```
if fewer than 3 snapshots in last 60 min:
```

```
confidence = min(confidence, 0.55) -> capped at Medium
```

Display: High (green) if >= 0.80, Medium (amber) if 0.55-0.79, Low (red) if < 0.55. Physical cash always has 0.95 confidence (no feed delays).

Stale Feed Detection

A feed is marked 'stale' when: `isStale = true AND latencyMs > 60,000ms`, OR last sync was more than 4 minutes ago. This is the Scenario C data-quality trigger. When stale, confidence drops, the card shows an amber 'stale' pill, and the system avoids giving confident recommendations.

Sparkline (60-min Trend)

Mini SVG chart showing the balance at each 5-minute snapshot over the last 60 minutes. Data source: `BalanceSnapshot` table, filtered by `agentId` and `scope` (`cash/bkash/nagad/rocket`). The line goes down when customers cash out, up when they cash in. An animated draw-line effect plays when the chart renders.

Aggregate Values

Total Value = Cash + all provider balances combined. Shared Cash = just the physical cash. E-Money = sum of all 3 provider balances. These are shown for awareness only — they do NOT mean balances are merged. Cross-provider conversion is explicitly forbidden.

8. Anomaly Detection Engine

Three lightweight, explainable anomaly detectors run across all agent/provider pairs every 35 seconds. Each returns human-readable evidence, uncertainty, a safe next step, and a documented false-positive note. An anomaly is NEVER declared as fraud.

Detector 1: Repeated Near-Identical Amounts

Detects when 5+ transactions of approximately the same amount (rounded to nearest 100 BDT) occur within 45 minutes from a small group of customers. Confidence rises with repetition and concentration but is capped at 0.78 because this pattern is common on Eid/salary days. Severity: High if ≤ 2 distinct customers, Warning otherwise.

Detector 2: Transaction Velocity Spike

Compares transaction count in the recent 30-minute window against the previous 30-minute window. Triggers if recent ≥ 10 transactions AND the ratio is $\geq 3.0\times$ the previous window. Confidence capped at 0.72. Always Warning severity (velocity alone is not suspicious).

Detector 3: Data-Quality Conflict

Detects stale or delayed provider feeds. Triggers when: `isStale = true` AND `latencyMs > 60s`, OR last sync > 4 minutes ago. Confidence reduced to max 0.45. The system avoids giving confident recommendations when data quality is poor. This implements the hackathon requirement: 'Show lower confidence or a safe fallback when data is missing, late, or conflicting.'

Suppression Window

If an alert of the same category + agent + provider was resolved within the last 15 minutes, the detector skips re-firing. This prevents the same pattern from immediately re-surfacing after a human reviews it.

Evidence Structure

Each alert includes a JSON evidence object with:

- `facts`: detected data points (amount, count, customers, window, total value)
- `sampleTx`: up to 4 sample transactions with ID, amount, customer, time
- `possibleNormalReasons`: legitimate explanations (Eid demand, merchant settlement, payday)
- `uncertainty`: why this might be wrong or inconclusive
- `safeNextStep`: what the reviewer should do (verify, contact, don't block)
- `falsePositiveNote`: expected FP risk level and why

9. AI Assistant & Advisory System

The platform uses OpenAI's API for three AI-powered features:

Feature	Model	How It Works
AI Assistant	GPT-4o-mini	Natural-language Q&A with conversation context. Gathers live network data (KPIs, alert
Alert Advisory	GPT-4o-mini	Generates structured advisory (situation, evidence, uncertainty, safe next step) in Beng
Voice Output (TTS)	OpenAI TTS-1	Converts AI assistant responses to speech (MP3 audio). 'Speak' button on each AI mes
Voice Input (ASR)	Web Speech API	Browser-native speech-to-text. Microphone button in the assistant input. Transcript auto

Safety Preamble

All AI calls include a strict system preamble that prohibits the words 'fraud', 'illegal', 'scam', or 'criminal', prohibits recommending blocking/freezing/accusing, requires careful language ('unusual', 'requires review'), preserves provider boundaries, and always notes that human review is required. If the OpenAI API is unavailable, all three features fall back to safe hand-authored templates.

10. Demo Scenarios (A/B/C/D)

Scenario	Title	What Happens	Role That Acts
A	Hidden Provider Shortage	Aggregate looks healthy but Nagad e-money at AG-KAR-001 acknowledges & escalates 1.5h	Risk Analyst, Field Ops
B	Liquidity + Unusual Activity	Cash draining fast + 6 repeated 4,900 BDT Rocket cash-outs from 2 customers	Risk Analyst, Field Ops
C	Data Inconsistency	bKash feed delayed 92s at AG-DHM-003. Confidence drops to 0.42 (Low). The system shows a data_quality alert and explicitly states: 'Confidence reduced; avoid confident recommendations.'	Field Ops
D	Coordinated Response	Nagad critically low at AG-GUL-002. Case routed, acknowledged, escalated, resolved with full audit trail	Area Manager

Scenario A Detail

Agent AG-KAR-001 (Karwan Bazar) appears healthy in aggregate: total value across cash + all providers is fine. But Nagad e-money is at 42,000 BDT (8% of 500,000 capacity), draining at 38,000 BDT/hour. The Unified Liquidity view shows a projected shortage card: '~1.5h to reach 20% threshold', severity: high, confidence: high. The shortage is hidden in the aggregate — only the per-provider view reveals it.

Scenario B Detail

Agent AG-MIR-004 (Mirpur) has physical cash draining AND 6 near-identical 4,900 BDT Rocket cash-outs from only 2 customers in ~40 minutes. The anomaly detector creates a 'repeated_amount' alert with full evidence. The Risk Analyst reviews: examines sample transactions, checks cross-provider patterns via the Relationship Graph, submits a Risk Review verdict. The pattern could be normal Eid demand (false-positive risk is HIGH) or may need field verification. The system never declares fraud.

Scenario C Detail

Agent AG-DHM-003 (Dhanmondi) has a delayed bKash feed: latency 92s, last sync 7 minutes ago, marked stale. Confidence drops to 0.42 (Low). The system shows a data_quality alert and explicitly states: 'Confidence reduced; avoid confident recommendations.' The Field Officer should wait for re-sync or cross-check with the agent before taking any action.

Scenario D Detail

Agent AG-GUL-002 (Gulshan) had a critical Nagad shortage. The case was auto-routed to Field Ops, acknowledged, escalated to Area Manager, and resolved via an authorised provider refill. The full audit trail shows every step with actor name, role, timestamp, and notes. The case is marked 'resolved' with a closing note. This scenario demonstrates the complete coordination workflow end-to-end.

11. Responsible Design Guardrails

The platform is designed to be advisory, not authoritative. It supports human decisions but never replaces them.

Principle	How It's Enforced
Careful language	Uses 'unusual', 'requires review' — never 'fraud'. AI preamble prohibits fraud-related words.
Human review required	Every alert routes to a human owner. No automatic actions.
No auto-blocking	Prototype never blocks, freezes, accuses, or moves funds.
Provider boundaries	Balances displayed separately, never merged. No cross-provider conversion supported.
Synthetic data only	No real customer identities, credentials, PINs, OTPs, or account data.
False-positive awareness	Each detector documents expected FP risk (e.g., 'HIGH on salary/Eid days').
Low-confidence fallback	Stale feeds reduce confidence. System avoids confident recommendations when data is poor.
Audit trail	Every action (ack, escalate, resolve, note, risk review) is logged with actor, role, timestamp.
Agent isolation	Agents can only see and act on their own outlet's data. Enforced at 3 levels.
Role separation	Risk Analyst reviews but cannot close. Agent flags but cannot resolve. Management is read-only.

12. Deployment Guide

Local Development

1. Install dependencies: `bun install` (or `npm install`)
2. Set `.env`: `DATABASE_URL=file:./db/custom.db + OPENAI_API_KEY=sk-...`
3. Push schema: `bun run db:push`
4. Start dev server: `bun run dev` (port 3000)
5. Start realtime: `cd mini-services/realtime && bun install && bun run dev` (port 3001)
6. The app auto-seeds on first API call if database is empty.

Vercel Deployment

1. Push to GitHub
2. Import on Vercel (auto-detects Next.js)
3. Set environment variables:
 - `DATABASE_URL` = `postgresql://...` (Neon Postgres connection string)
 - `OPENAI_API_KEY` = `sk-...` (your OpenAI key)
4. `vercel.json` runs: `prisma generate + prisma db push + next build`
5. Deploy: `vercel --prod`
6. Database auto-seeds on first API call.
7. Realtime service does not run on Vercel (serverless). Deploy separately on Railway or accept manual refresh.

Environment Variables

Variable	Required	Example
<code>DATABASE_URL</code>	Yes	<code>postgresql://user:pass@host/db?sslmode=require</code> (Vercel) or <code>file:./db/custom.db</code> (local)
<code>OPENAI_API_KEY</code>	Yes (for AI)	<code>sk-proj-xxxxxxxxxxxxxxxxxxxxxx</code>

SALI — Super Agent Liquidity & Risk Intelligence Platform
Codex Community Hackathon | bKash presents SUST CSE Carnival 2026
All data is SYNTHETIC. No real wallets, credentials, or customer data.